5. Análisis y reflexión

1. Tabla de complejidades

Nº	**Problema**	**Estructura Principal**	**Complejidad Temporal (Big-O)**	**Complejidad Espacial**
#232	Implement Queue using Stacks	Dos Pilas (Stacks)	Amortizado $\mathcal{O}(1)$ / Peor caso $\mathcal{O}(n)$ en pop/peek; $\mathcal{O}(1)$ en push	$\mathcal{O}(n)$
#622	Design Circular Queue	Arreglo de tamaño fijo / Lista	$\mathcal{O}(1)$ para todas las operaciones (enqueue, dequeue, front, rear)	$\mathcal{O}(k)$
#239	Sliding Window Maximum	Cola de doble extremo (Deque) / Monotonic Queue	$\mathcal{O}(n)$	$\mathcal{O}(k)$

2. Reflexión

La diferencia entre listas y deques

El costo de `list.pop(0)` es $\mathcal{O}(n)$ porque las listas en Python se implementan como arreglos dinámicos contiguos en memoria. Al eliminar el primer elemento, todos los elementos restantes deben desplazarse una posición a la izquierda para mantener la contigüidad, lo que requiere un tiempo proporcional al tamaño de la lista. En contraste, `deque.popleft()` de la biblioteca `collections` es $\mathcal{O}(1)$ debido a que está implementada como una lista doblemente enlazada (o de bloques enlazados). Para eliminar el elemento inicial, solo se requiere reasignar los punteros del nodo cabeza y liberar la memoria del nodo descartado, una operación de tiempo constante que no afecta al resto de los elementos.

Dificultades en la elección de la estructura

Elegir la estructura adecuada para el problema **Sliding Window Maximum (#239)** fue el mayor reto para el equipo. Inicialmente, la tentación de usar una solución de fuerza bruta con $\mathcal{O}(n \cdot k)$ fue descartada al analizar los límites de tiempo. Tras debatir y contrastar opciones, determinamos que una **cola de doble extremo (deque)** con comportamiento monótonamente decreciente era la ideal. El desafío consistió en entender cómo mantener los índices dentro de la ventana de forma que el elemento máximo siempre estuviera al frente. Esta discusión nos permitió comprender la importancia de optimizar tanto el tiempo como el espacio mediante la estructura correcta.

Uso de `re.fullmatch` sobre `re.match`

La ventaja de `re.fullmatch` sobre `re.match` es que exige que la totalidad de la cadena cumpla con el patrón, desde el inicio hasta el final, mientras que `re.match` solo busca que el patrón coincida a partir del primer carácter, ignorando lo que siga. Por ejemplo, al validar un código postal de 5 dígitos con el patrón `r'\d{5}'`, `re.match` consideraría válida la cadena `'12345-extra'` porque encuentra los 5 dígitos al comienzo. En cambio, `re.fullmatch` rechazaría `'12345-extra'` al no acoplarse por completo, garantizando que el dato sea exactamente el formato requerido sin añadidos.